

Search Algorithms using Heuristics

Informed Search

Heuristic Search

TextBook: R&N 3rd Edition Section 3.5

Prof. Chris Ding

Lecture slides for Prof. C. Ding's lecture. Not allowed to pass on to others.

When doing BSF, DFS, Dijkstra algorithms, we did not use any information beyond the connectivity between graph nodes. These algorithms are therefore call **blind search** algorithms.

In many situations, we have **some additional information** (often inaccurate). This can help speedup the search process. These information is called **heuristics**. **Although the heuristics are inaccurate, approximate, they are often very helpful.**

Use a heuristic (estimate) function $h(n)$

$h(n)$ =**estimated** cost from current node to goal node(n)

$g(n)$ =**true** cost from the start node to current node n

Simple: Greedy, Best First Algorithm:

$f(n) = h(n)$ % current $\rightarrow$ goal

Advanced: A* algorithm

$f(n) = g(n) + h(n)$ % start $\rightarrow$ current + current $\rightarrow$ goal

What are Heuristics

- Rule of thumb, intuition
- A **quick way** to estimate how close we are to the goal. How close is a state to the goal..
- Pearl: "the ever-amazing observation of how much people can accomplish with that simplistic, unreliable information source known as *intuition*."

8-puzzle

- $h_1(n)$: number of misplaced tiles
- $h_2(n)$: Manhattan distance

7	2	4
5		6
8	3	1

Start State

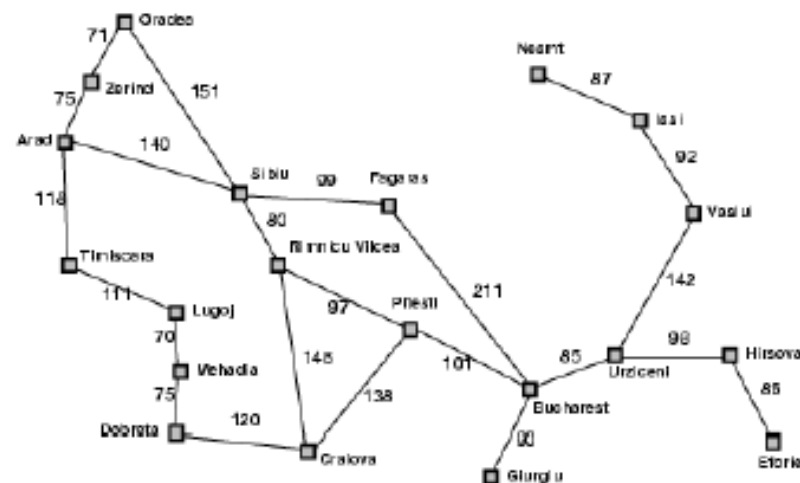
	1	2
3	4	5
6	7	8

Goal State

$$h_1(S) = 8$$

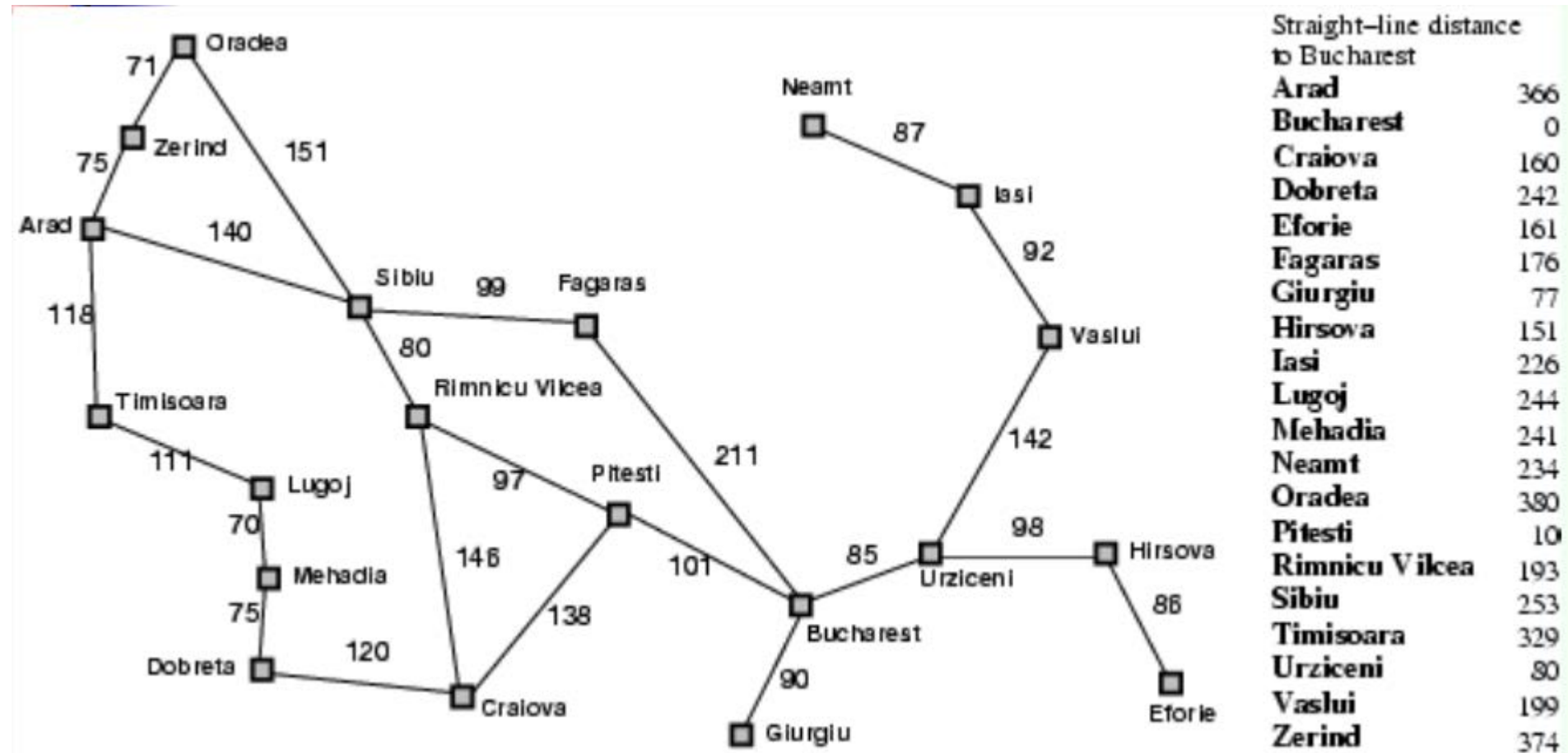
$$h_2(S) = 3+1+2+2+3+2+2+3 = 18$$

- Path-finding on a map
 - Euclidean distance



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Cluj	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	300
Pitesti	101
Rimnicu Vilcea	193
Sibiu	253
Timisoara	319
Urziceni	80
Vaslui	199
Zerind	374

$F(\text{node}) = \text{straight-line-distance}(\text{node}, \text{Bucharest})$

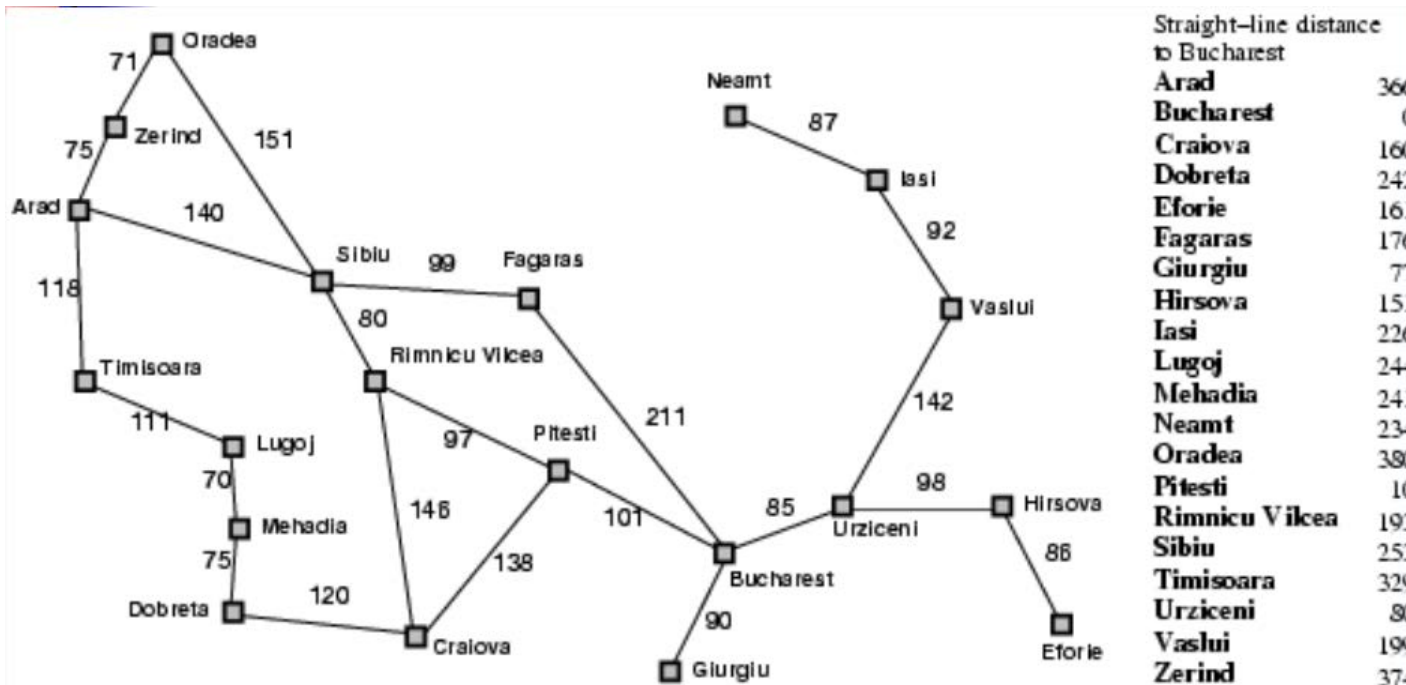
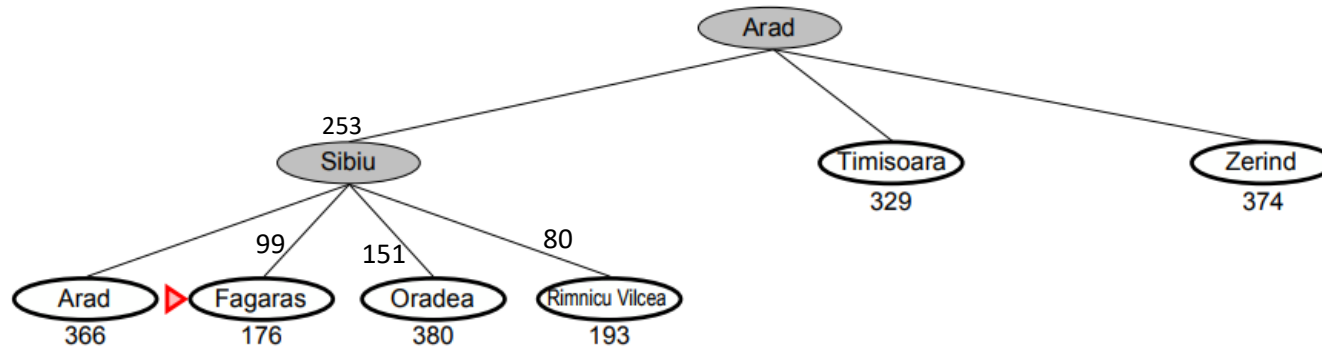


Greedy (best-first) Search

$$F(n) = \text{dist-to-Bucharest}(n)$$

Greedy search example

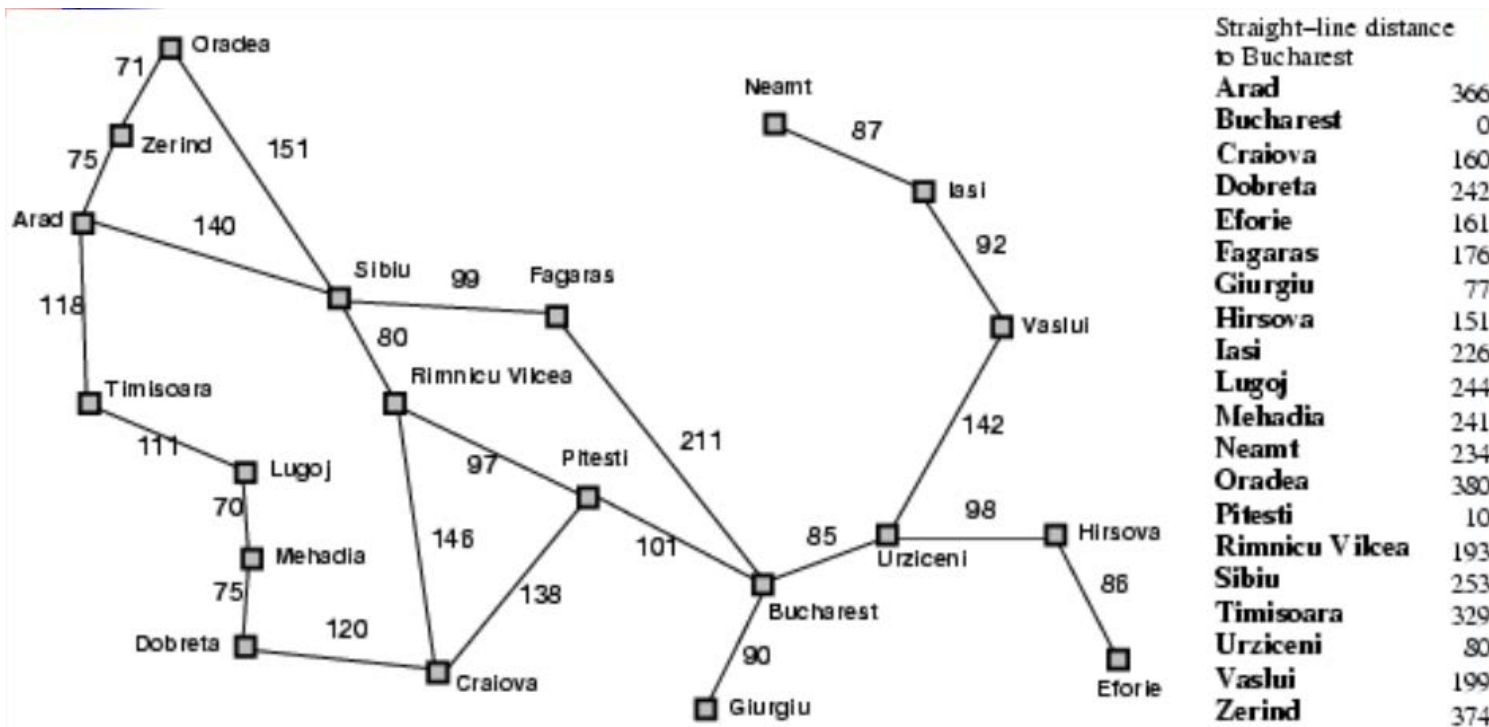
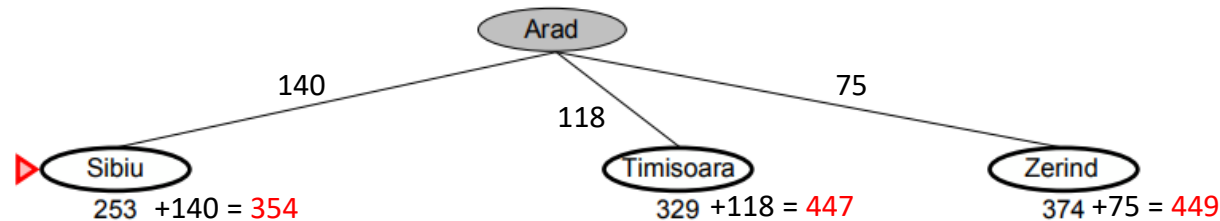
n: next node



A* Search

$$F(n) = \text{dist}(n_0, n) + \text{dist}(n, \text{Bucharest})$$

$$F(n) = \text{true-cost}(n_0, n) + \text{estimate-dist}(n, \text{Bucharest})$$

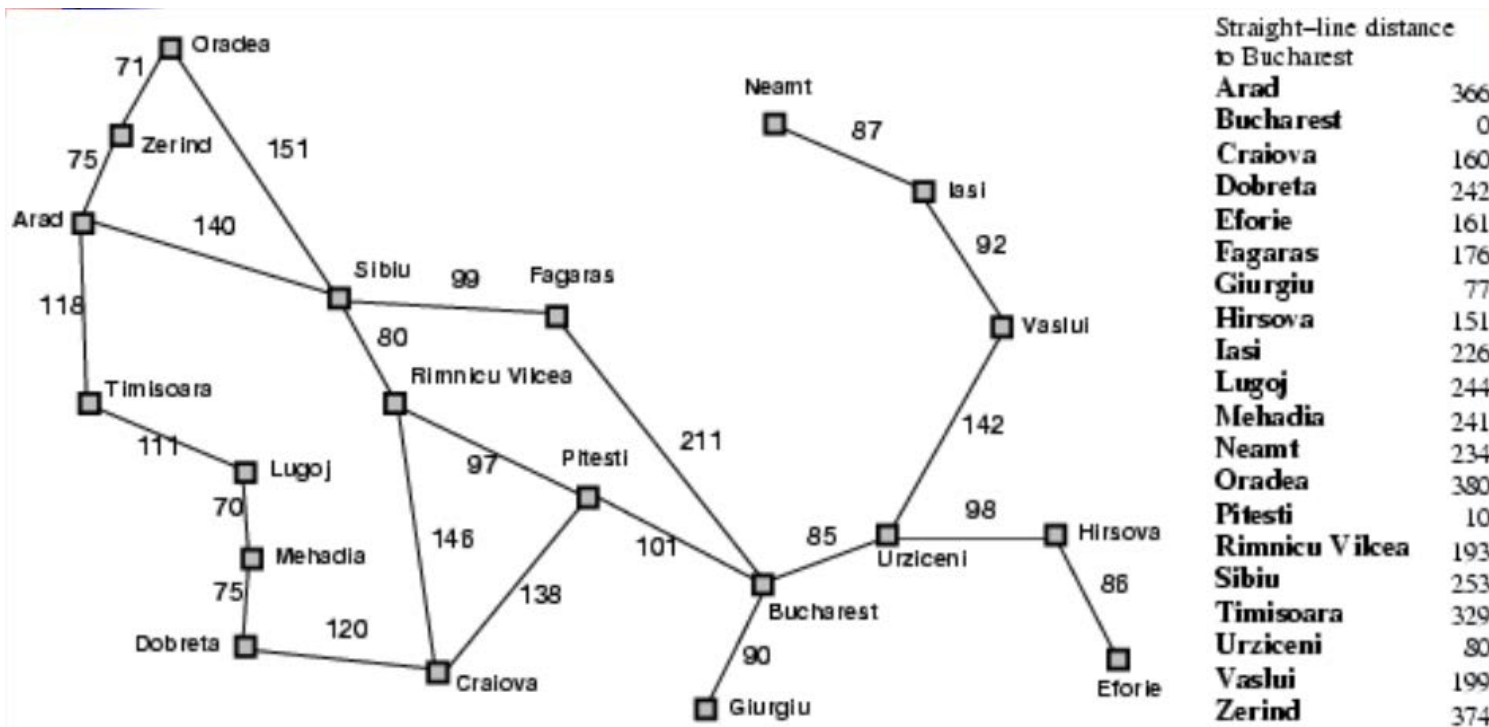
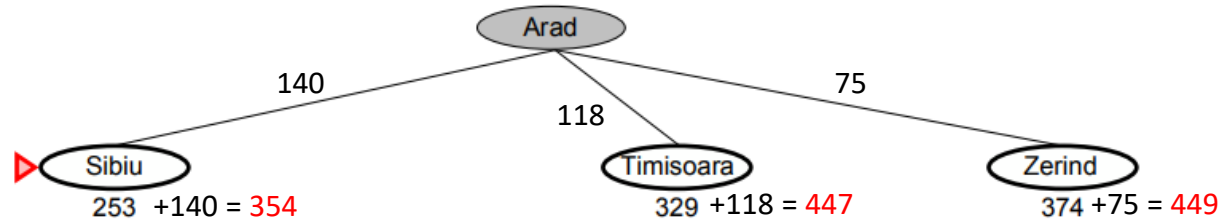


A* Search

$$F(n) = \text{true-dist}(n_0, n) + \text{estimated-dist}(n, \text{Bucharest})$$

n_0 : current node

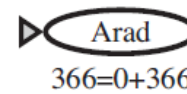
n : next node



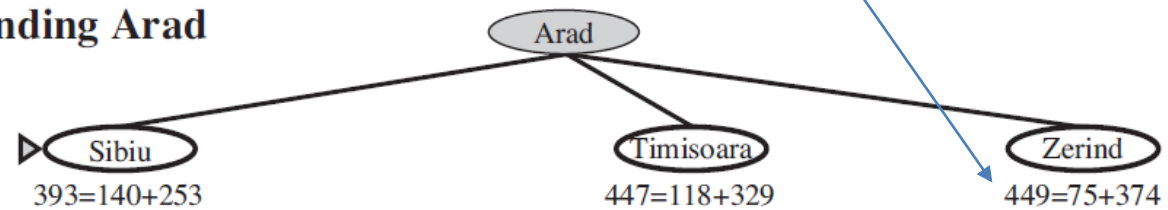
A* Search

$f(n)=g(n)+h(n)$ TextBook: p.96

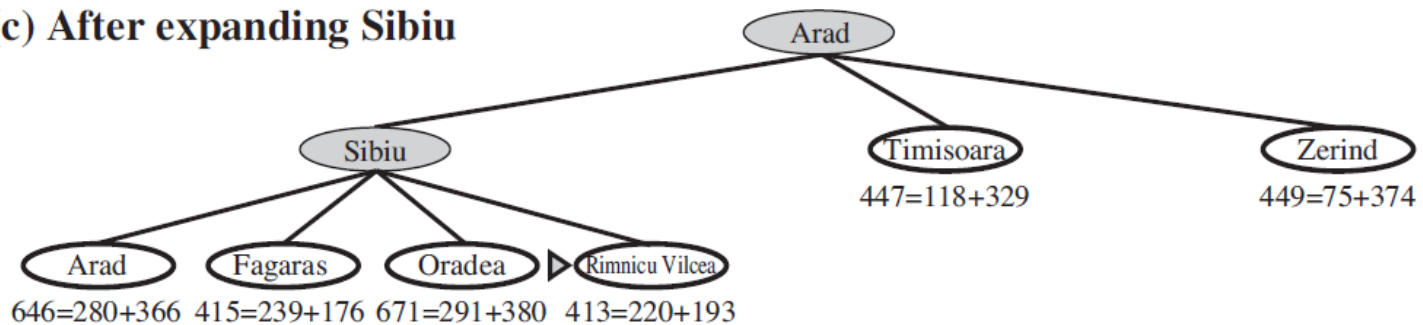
(a) The initial state



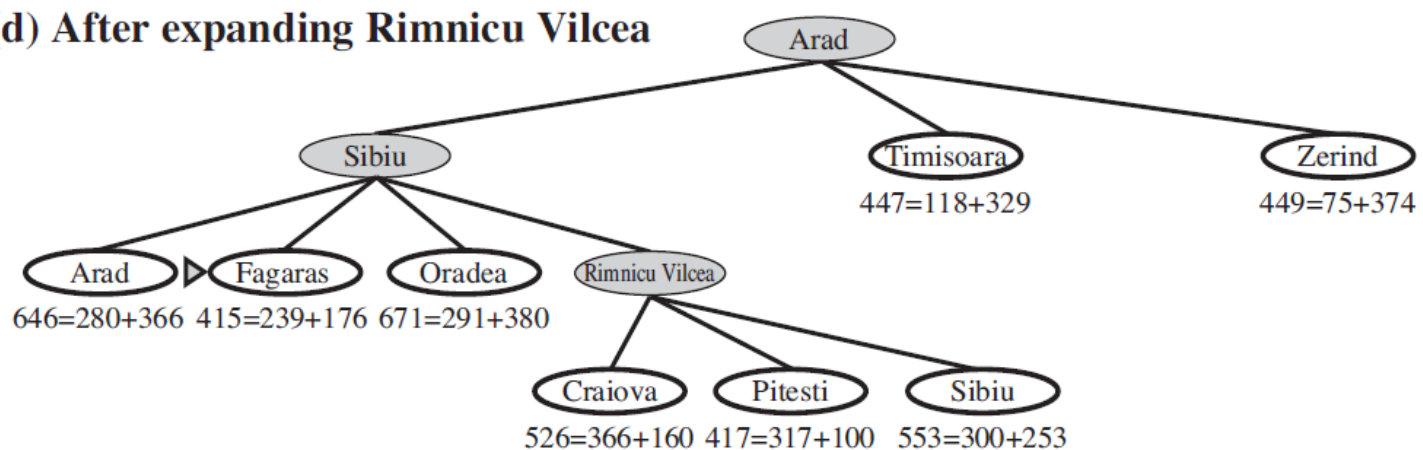
(b) After expanding Arad



(c) After expanding Sibiu



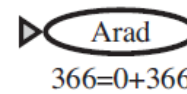
(d) After expanding Rimnicu Vilcea



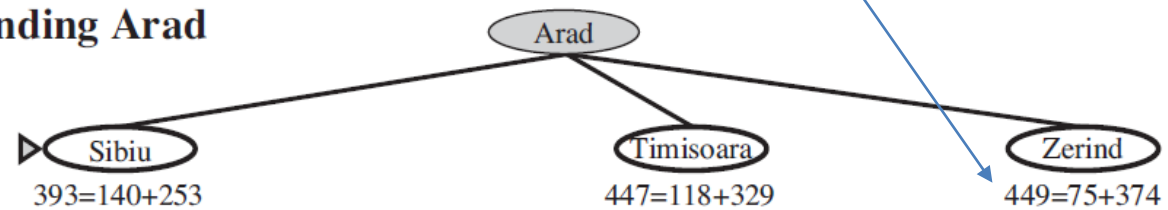
A* Search

$f(n)=g(n)+h(n)$ TextBook: p.96

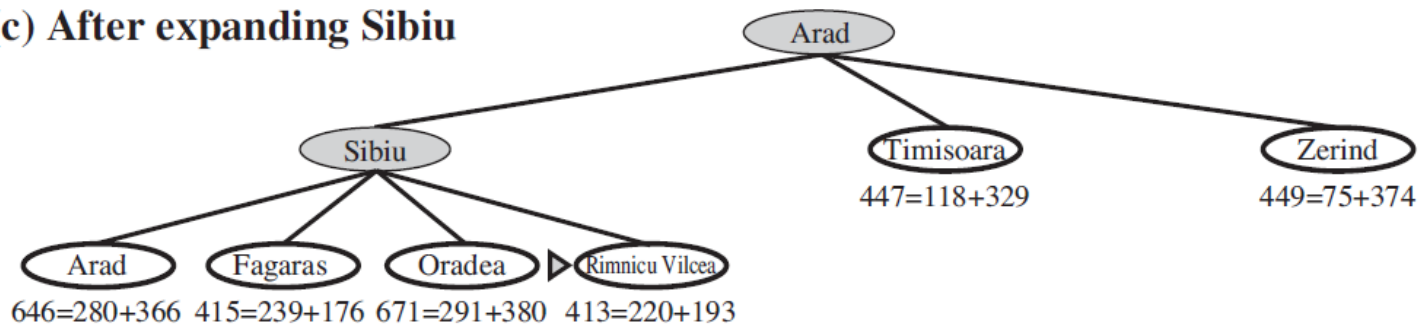
(a) The initial state



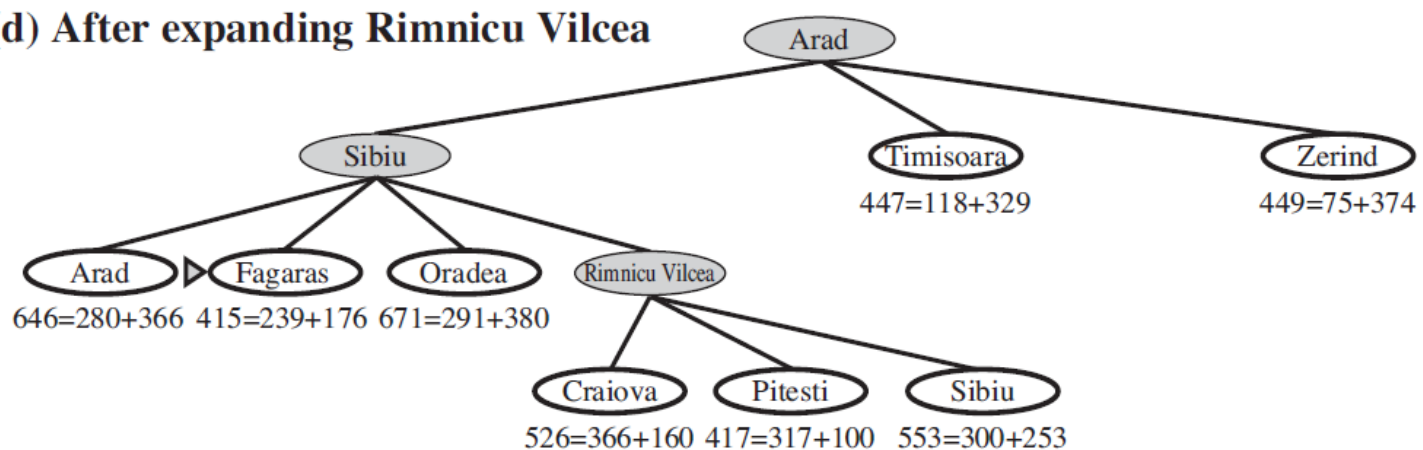
(b) After expanding Arad



(c) After expanding Sibiu



(d) After expanding Rimnicu Vilcea



各大城市到台湾的距离

乌鲁木齐

A

$h=30$

17

$h=27$

西宁

X

11

$h=33$

拉萨

L

20

成都

C

$h=17$

香港

K

$h=19$

武汉

W

$h=9$

12

北京

B

$h=16$

16

$h=22$

H

哈尔滨

6

沈阳

S

$h=17$

台湾

T

$h=0$

17

9

10

11

12

16

17

30

33

19

22

17

16

12

11

10

9

8

7

6

5

4

3

2

1

0

-1

-2

-3

-4

-5

-6

-7

-8

-9

-10

-11

-12

-13

-14

-15

-16

-17

-18

-19

-20

-21

-22

-23

-24

-25

-26

-27

-28

-29

-30

-31

-32

-33

-34

-35

-36

-37

-38

-39

-40

-41

-42

-43

-44

-45

-46

-47

-48

-49

-50

-51

-52

-53

-54

-55

-56

-57

-58

-59

-60

-61

-62

-63

-64

-65

-66

-67

-68

-69

-70

-71

-72

-73

-74

-75

-76

-77

-78

-79

-80

-81

-82

-83

-84

-85

-86

-87

-88

-89

-90

-91

-92

-93

-94

-95

-96

-97

-98

-99

-100

-101

-102

-103

-104

-105

-106

-107

-108

-109

-110

-111

-112

-113

-114

-115

-116

-117

-118

-119

-120

-121

-122

-123

-124

-125

-126

-127

-128

-129

-130

-131

-132

-133

-134

-135

-136

-137

-138

-139

-140

-141

-142

-143

-144

-145

-146

-147

-148

-149

-150

-151

-152

-153

-154

-155

-156

-157

-158

-159

-160

-161

-162

-163

-164

-165

-166

-167

-168

-169

-170

-171

-172

-173

-174

-175

-176

-177

-178

-179

-180

-181

-182

-183

-184

-185

-186

-187

-188

-189

-190

-191

-192

-193

-194

-195

-196

-197

-198

-199

-200

-201

-202

-203

-204

-205

-206

-207

-208

-209

-210

-211

-212

-213

-214

-215

-216

-217

-218

-219

-220

-221

-222

-223

-224

-225

-226

-227

-228

-229

-230

-231

-232

-233

-234

-235

-236

-237

-238

-239

-240

-241

-242

-243

-244

-245

-246

-247

-248

-249

-250

-251

-252

-253

-254

-255

-256

-257

-258

-259

-260

-261

-262

-263

-264

-265

-266

-267

-268

-269

-270

-271

-272

-273

-274

-275

-276

-277

-278

-279

-280

-281

-282

-283

-284

-285

-286

-287

-288

-289

-290

-291

-292

-293

-294

-295

-296

-297

-298

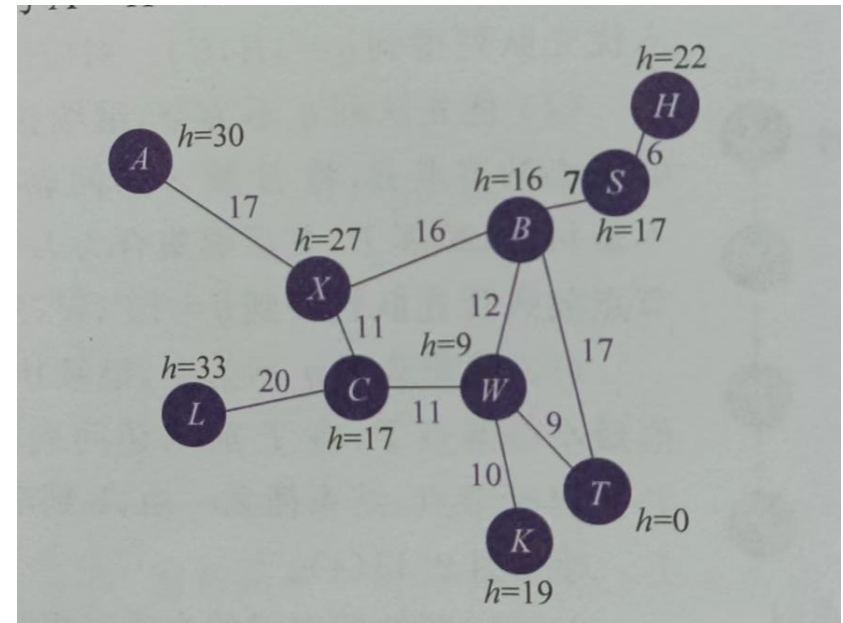
-299

-300

-301

Simple example of Best First search and A* Search

Start node is A, goal node is T



Best First Search: 1. Only choice is A $\rightarrow$ X.

2. Two choices: $h(B)=16$, $h(C)=17$. Choose B.

3. $h(T)=0$. Choose T. **STOP**

A $\rightarrow$ X $\rightarrow$ B $\rightarrow$ T

cost= $17+16+17=50$

A* Search: 1. Only choice is A $\rightarrow$ X.

2. Two choices: $17+16+h(B) = 49$, $17+11 + h(C)=45$. Choose C.

3. Two choices: $g(c)+20+h(L) = 53$, $g(c)+11+h(W)=20$. Choose W.

4. Three choices: $10+h(K)=29$, $12+h(B)=29$, $9+h(T) = 9$. Choose T

A $\rightarrow$ X $\rightarrow$ C $\rightarrow$ W $\rightarrow$ T

cost= $17+11+11+9=48$

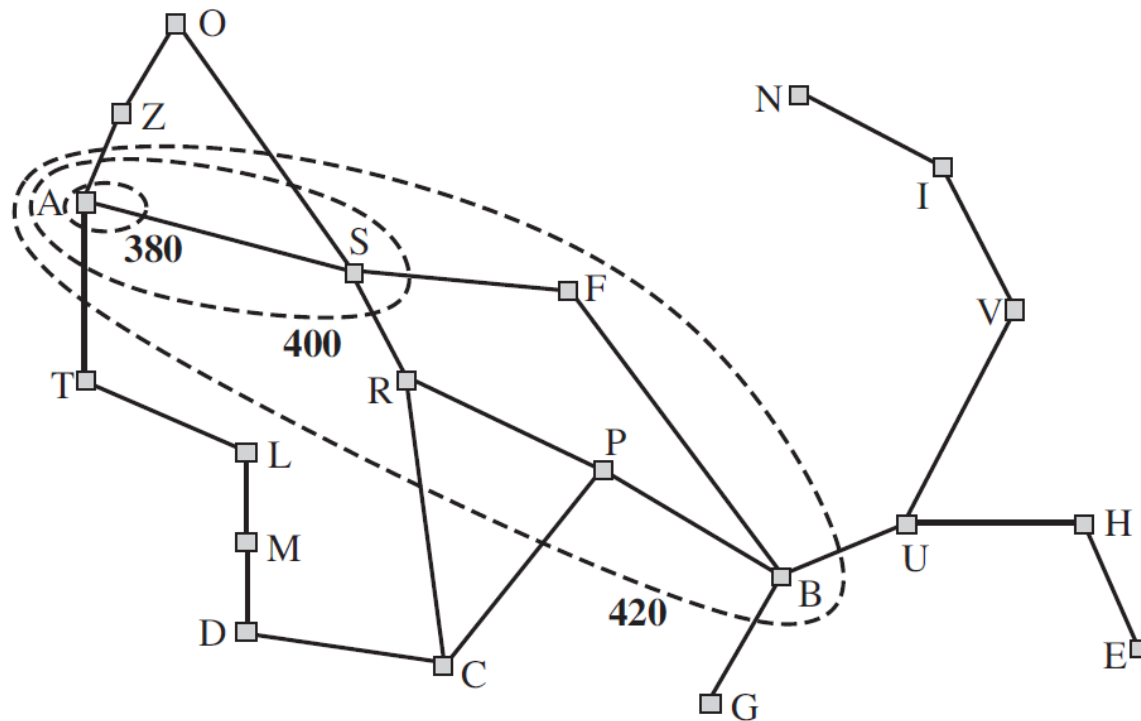
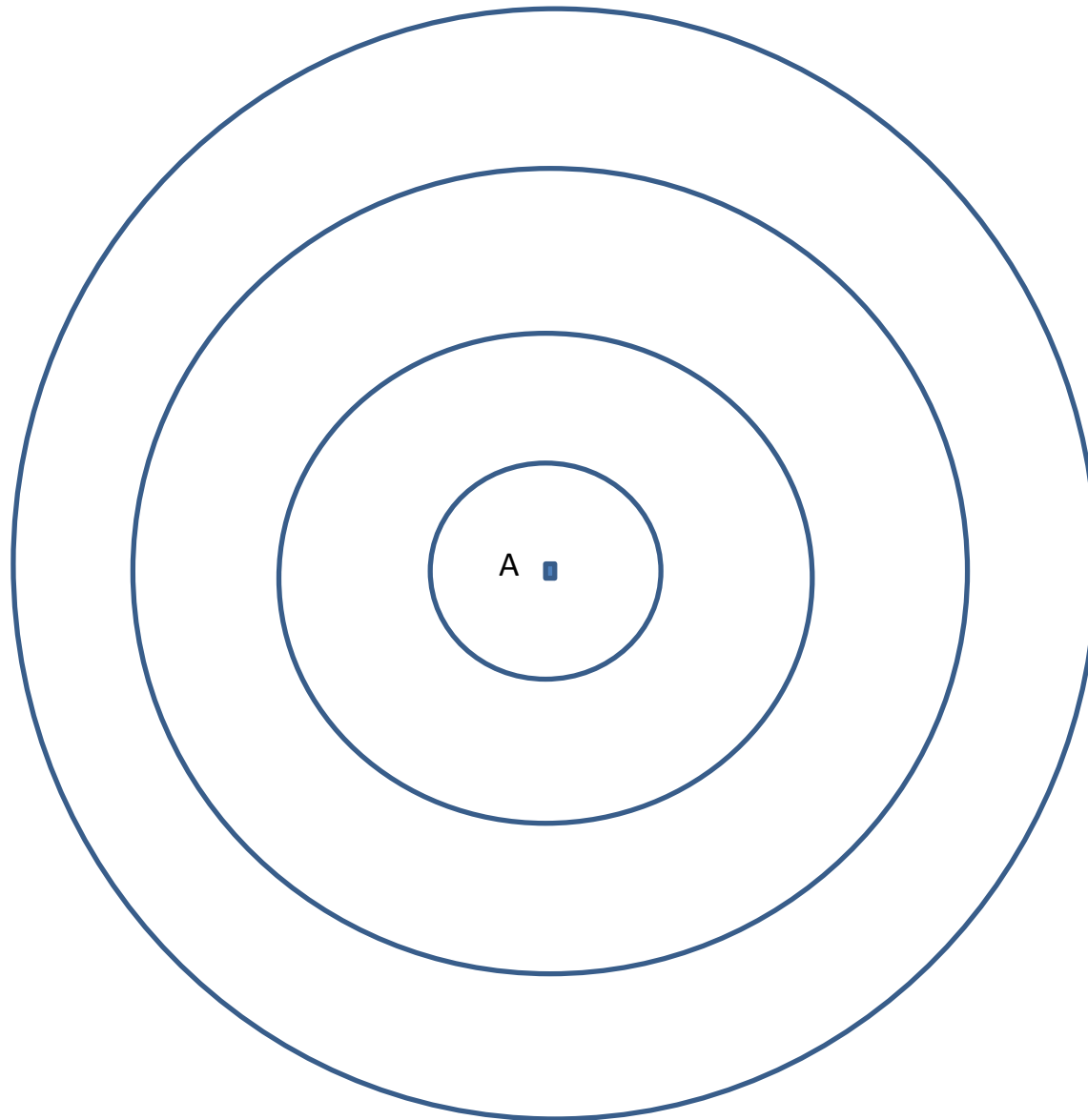


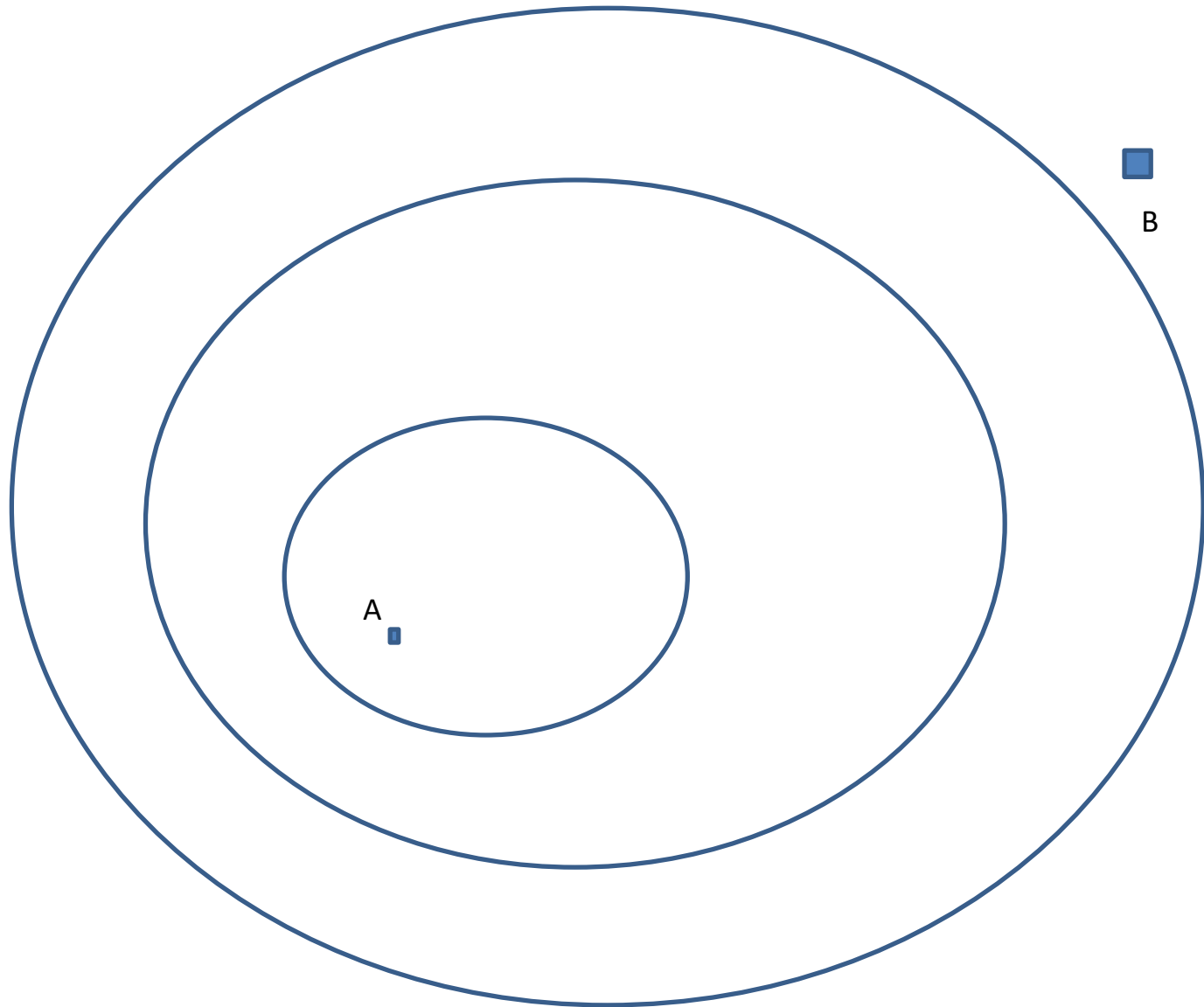
Figure 3.25 Map of Romania showing contours at $f = 380$, $f = 400$, and $f = 420$, with Arad as the start state. Nodes inside a given contour have f -costs less than or equal to the contour value.

Run Dijkstra algorithm

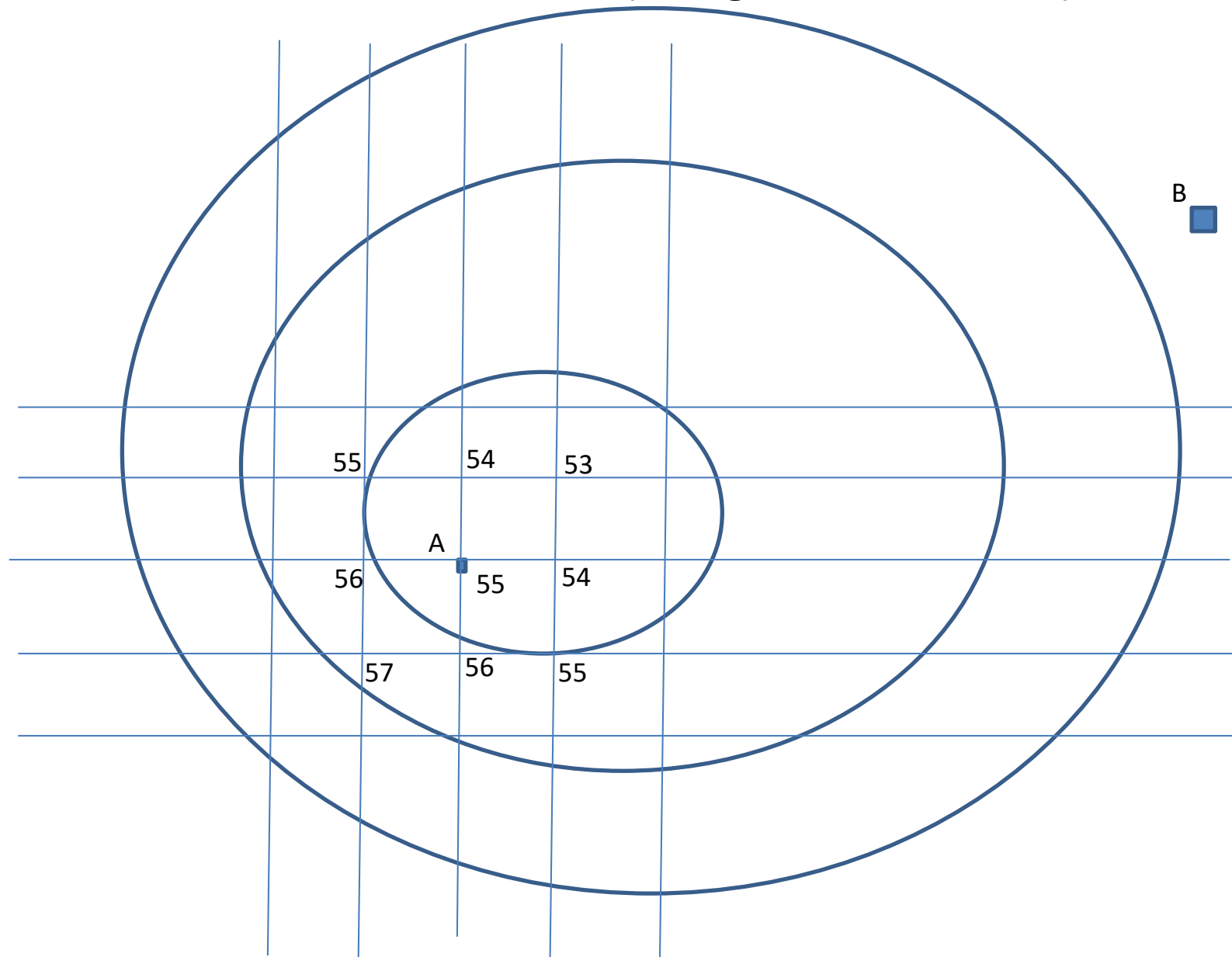


B

Run A* Search algorithm



Heuristic score (straight distance to B)



Standard description:

Textbook: p.92

Use a heuristic (estimate) function $h(n)$

$h(n)$ =cost from current node to goal node $(n) = \text{cost}(n_current, n_goal)$

$g(n)$ =cost from the start node to current node $n = \text{cost}(n_start, n_current)$

Best First (greedy) Algorithm:

$$f(n) = h(n)$$

A* algorithm

$$f(n) = g(n) + h(n)$$

Easier description:

n_0 =start node, n_goal =goal node, $n_current$ =a node on the path

$h(n)$ =estimated-cost ($n_current$, n_goal)

$g(n)$ =cost-so-far($n_0, n_current$)

Best First (greedy) Algorithm:

$$f(n) = h(n) = \text{estimated-cost}(n, n_goal)$$

A* algorithm

$$f(n) = g(n) + h(n) = \text{cost-so-far}(n_0, n) + \text{estimated-cost}(n, n_1)$$

Optimal Properties of A* Search

Admissible heuristic:

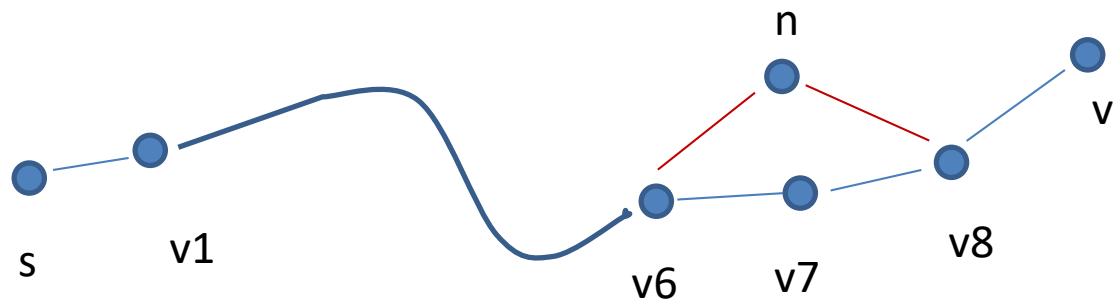
$$h(n) \leq \text{true cost from } n \text{ to goal}$$

Thm1: A* finds the optimal path if $h(n)$ is admissible.

Heuristics satisfy **Consistency**:

$$h(n_i) \leq h(n_j) + \text{cost}(n_i, n_j)$$

Thm2: A* find the optimal path if $h(n)$ is consistent



Optimal Properties of A* Search

Admissible heuristic: $h(n) \leq \text{true cost from } n \text{ to goal}$

Theorem 1: A* finds the optimal path if $h(n)$ is admissible.

Proof. We use prove by contradiction.

Suppose the optimal cost is C^* . But the cost C of the path obtained from A* is bigger: $C > C^*$.

Assume the non-optimal path is same as an optimal path $(s, v1, \dots, v6, v7, v8, v)$ but only node n is not on the path.

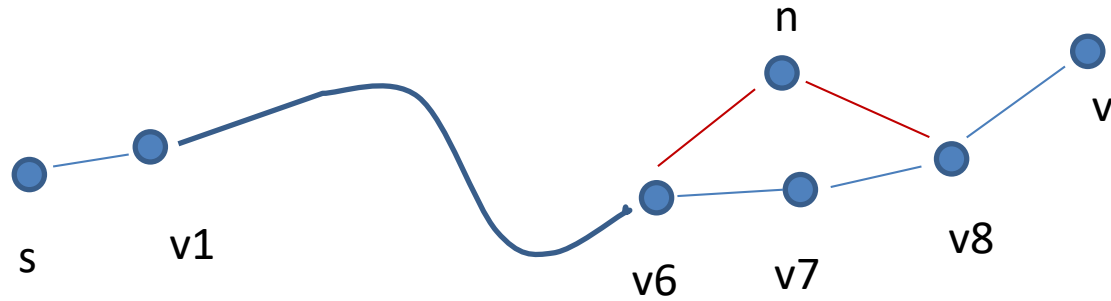
$$C(n) = g(n) + h(n)$$

$$C(n) = g^*(n) + h(n)$$

$$C(n) \leq g^*(n) + h^*(n) \text{ , admissibility}$$

$$C(n) \leq C^*$$

This contradicts to $C > C^*$.



Optimal Properties of A* Search

Consistency:

$$h(n_i) \leq h(n_j) + \text{cost}(n_i, n_j)$$

Theorem 2: A* find the optimal path if $h(n)$ is consistent

Proof.

A1. If $h(n)$ is increasing (non-decreasing), the cost much be increasing (non-decreasing) along any path from start to goal. Assume n_j is next after n_i .

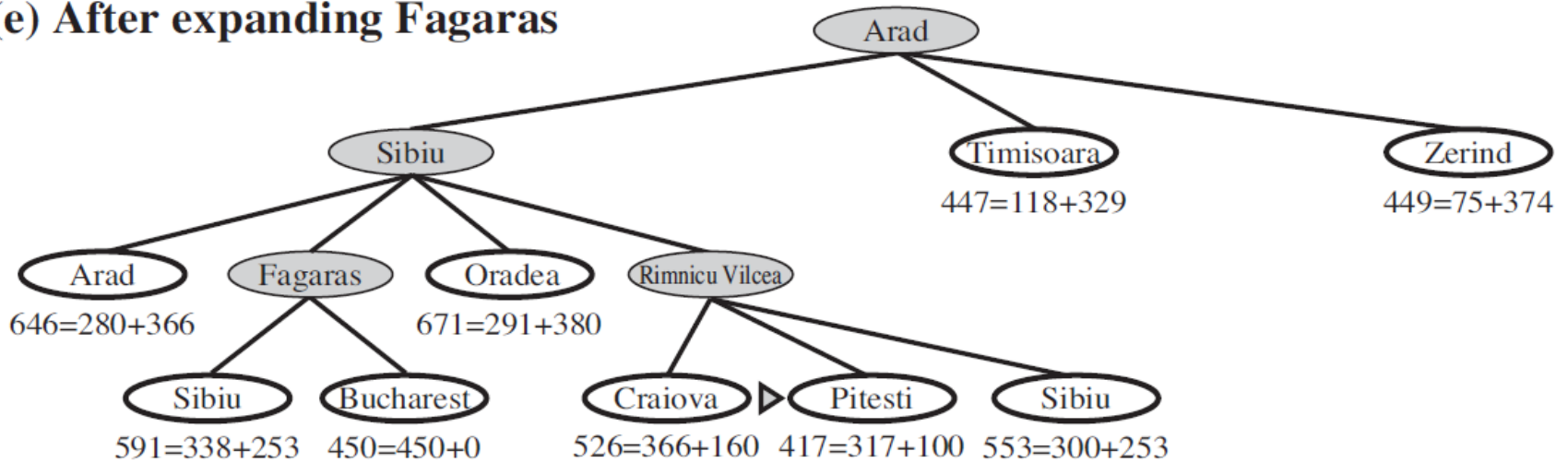
$$f(n_j) = g(n_j) + h(n_j) = [g(n_i) + \text{cost}(n_i, n_j)] + h(n_j) \geq g(n_i) + h(n_j).$$

A2. When A* reaches n_i , the path from n_{start} to n_i is already an optimal path. The is the nature of greedy search.

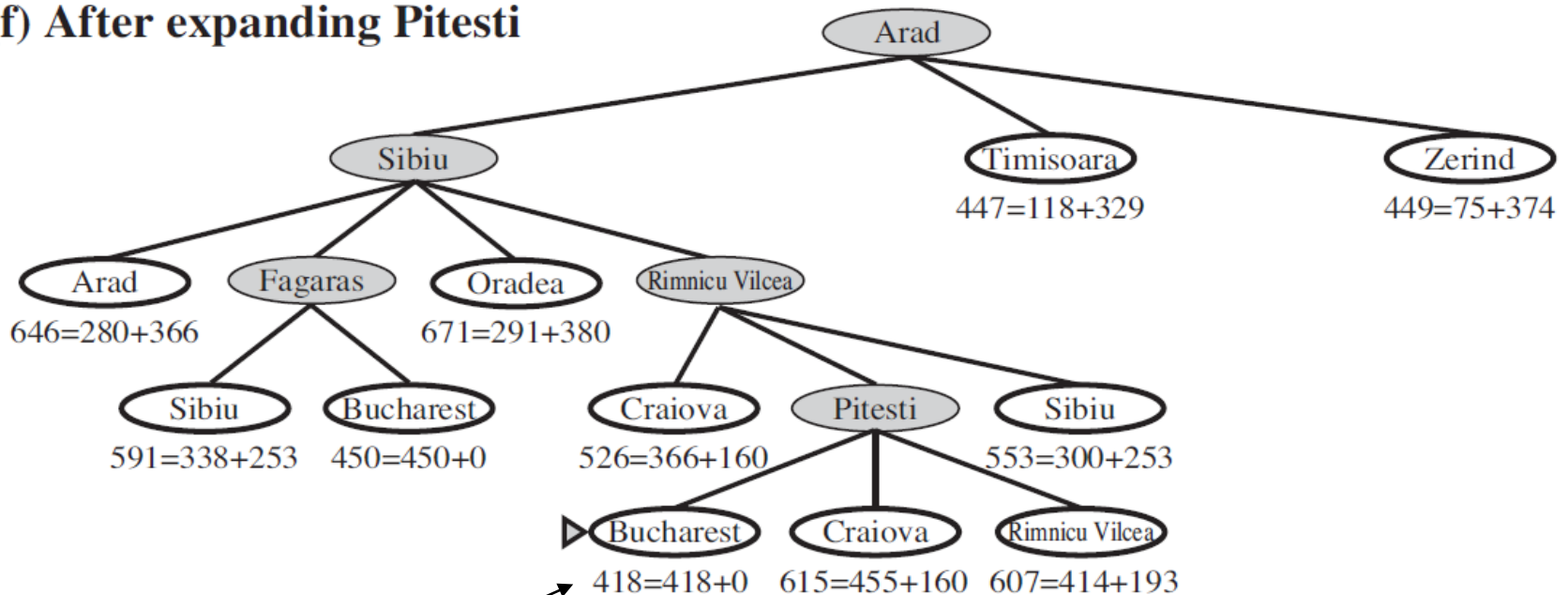
A* Search

$f(n)=g(n)+h(n)$ TextBook: p.96

(e) After expanding Fagaras

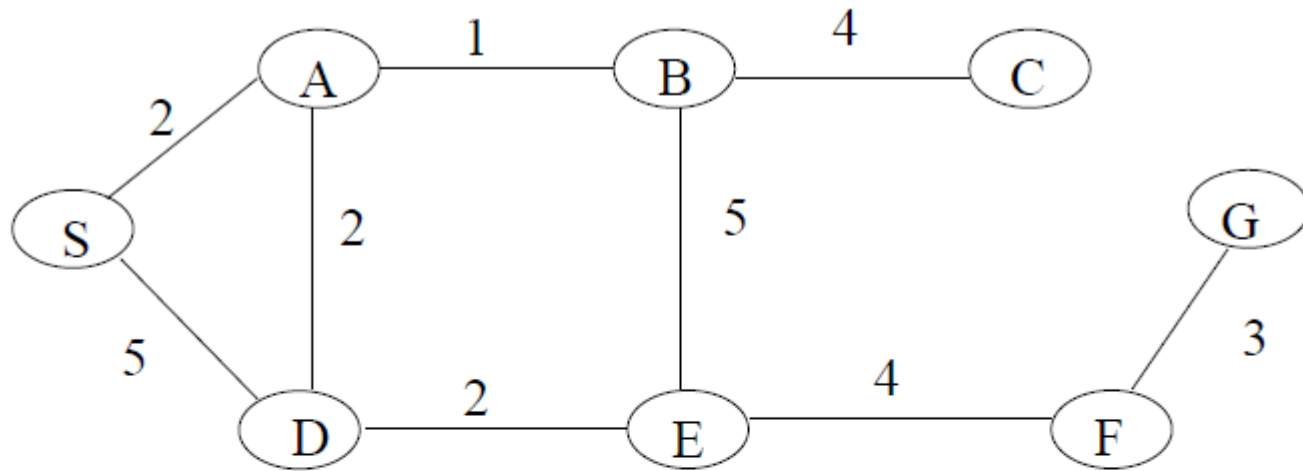


(f) After expanding Pitesti

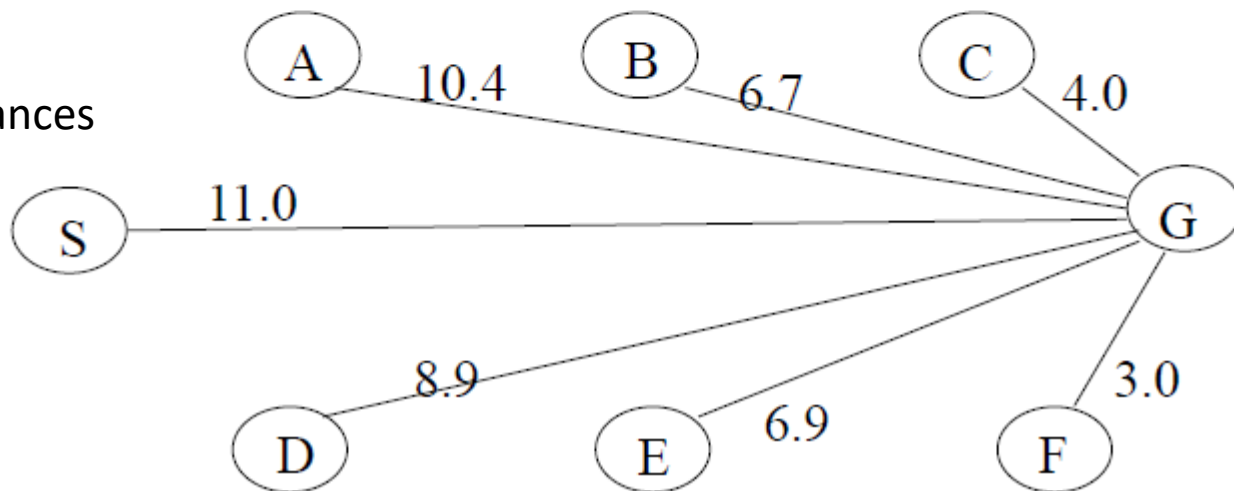


Shortest distance

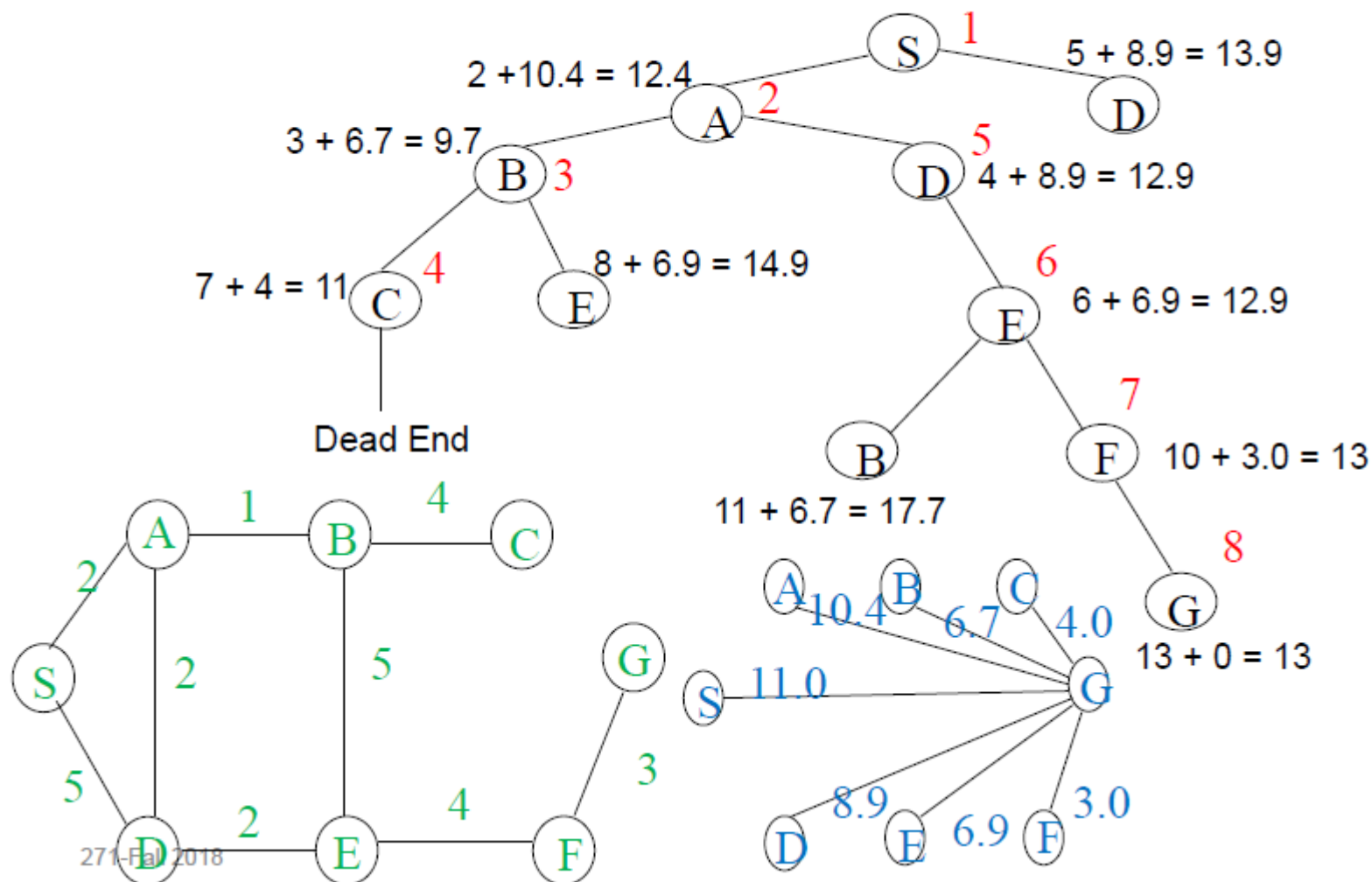
Another A* Search Example

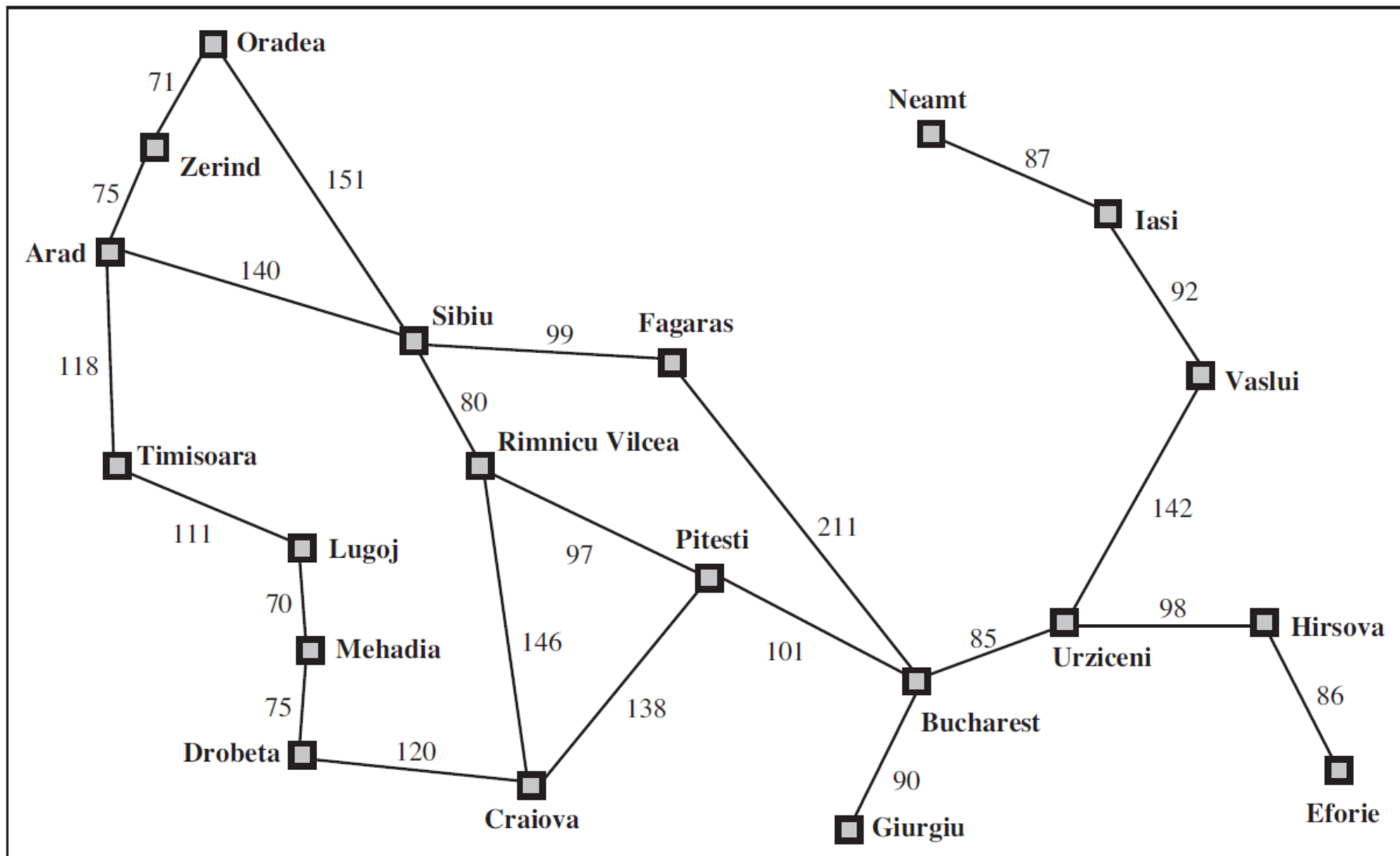


Direct distances



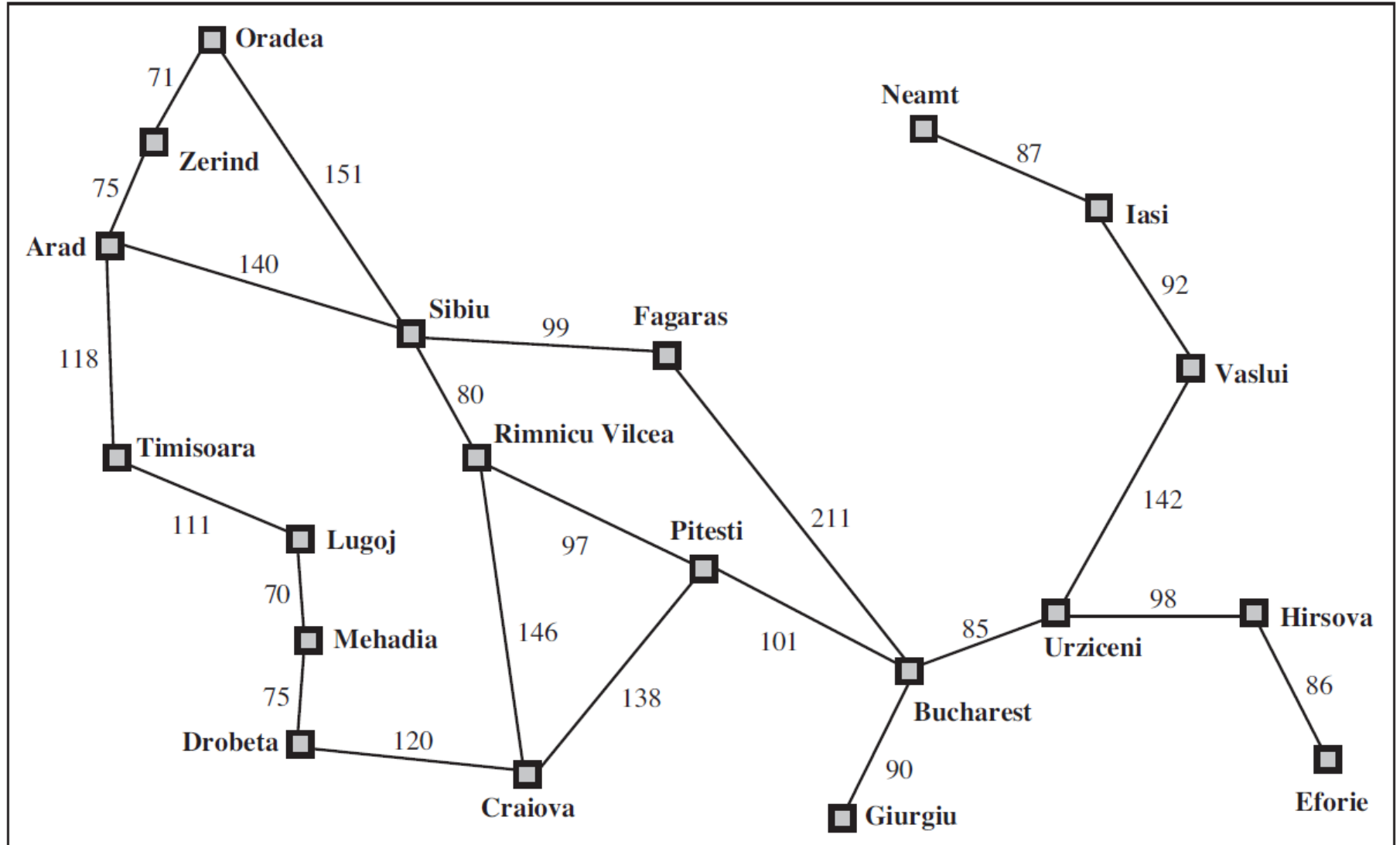
Example of A* Algorithm in Action

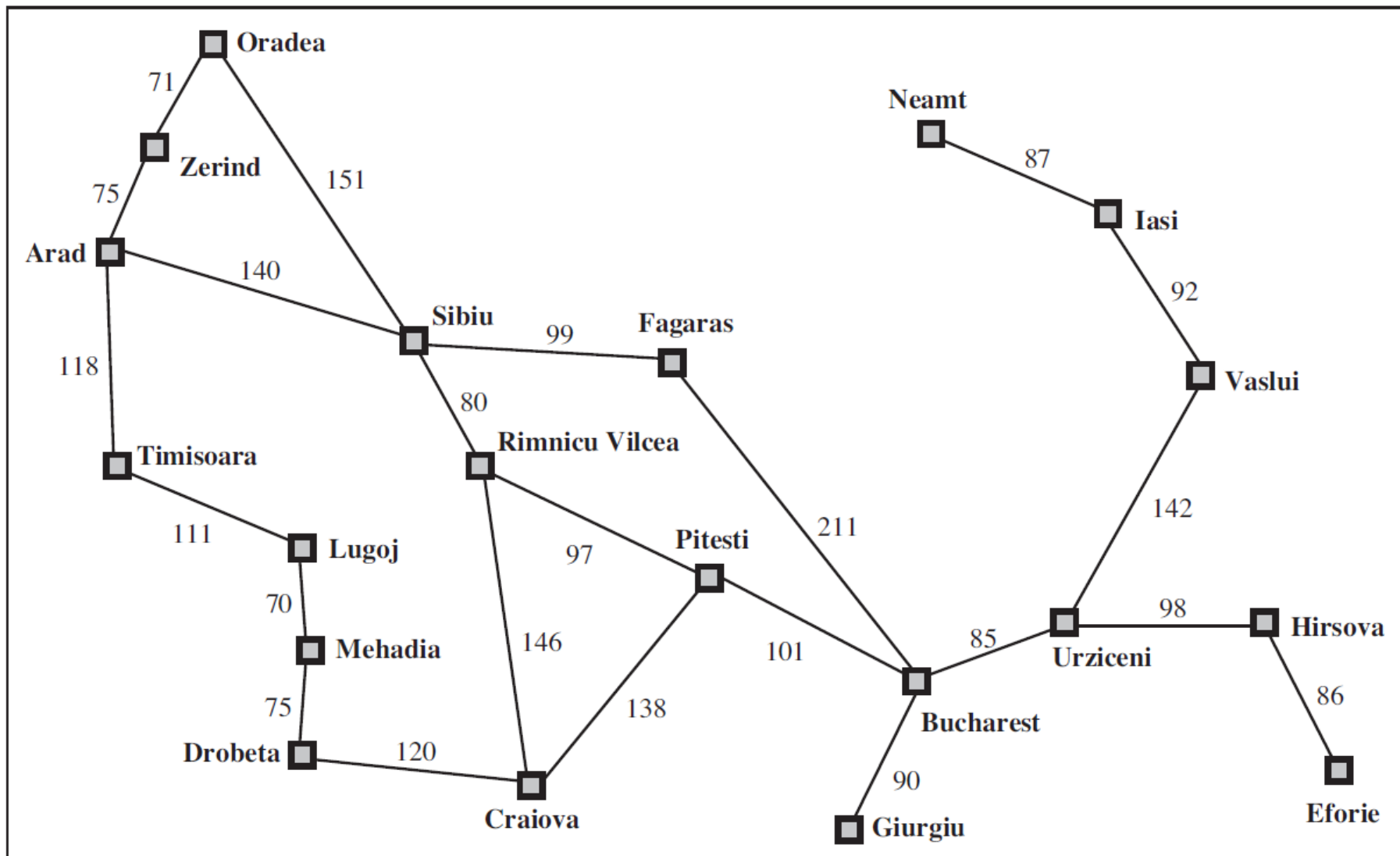




Dijkstra Algorithm steps

Arad to Bucharest





What are Heuristics

- Rule of thumb, intuition
- A **quick way** to estimate how close we are to the goal. How close is a state to the goal..
- Pearl: "the ever-amazing observation of how much people can accomplish with that simplistic, unreliable information source known as *intuition*."

8-puzzle

- $h_1(n)$: number of misplaced tiles
- $h_2(n)$: Manhattan distance

7	2	4
5		6
8	3	1

Start State

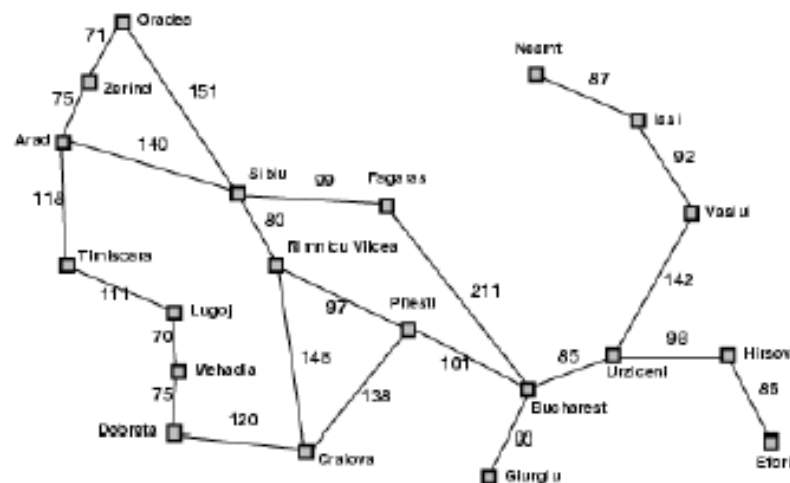
	1	2
3	4	5
6	7	8

Goal State

$$h_1(S) = 8$$

$$h_2(S) = 3+1+2+2+3+2+2+3 = 18$$

- Path-finding on a map
 - Euclidean distance



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Cluj	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	101
Rimnicu Vilcea	193
Sibiu	253
Timisoara	319
Urziceni	80
Vaslui	199
Zerind	374

$$f(n) = h(n)$$

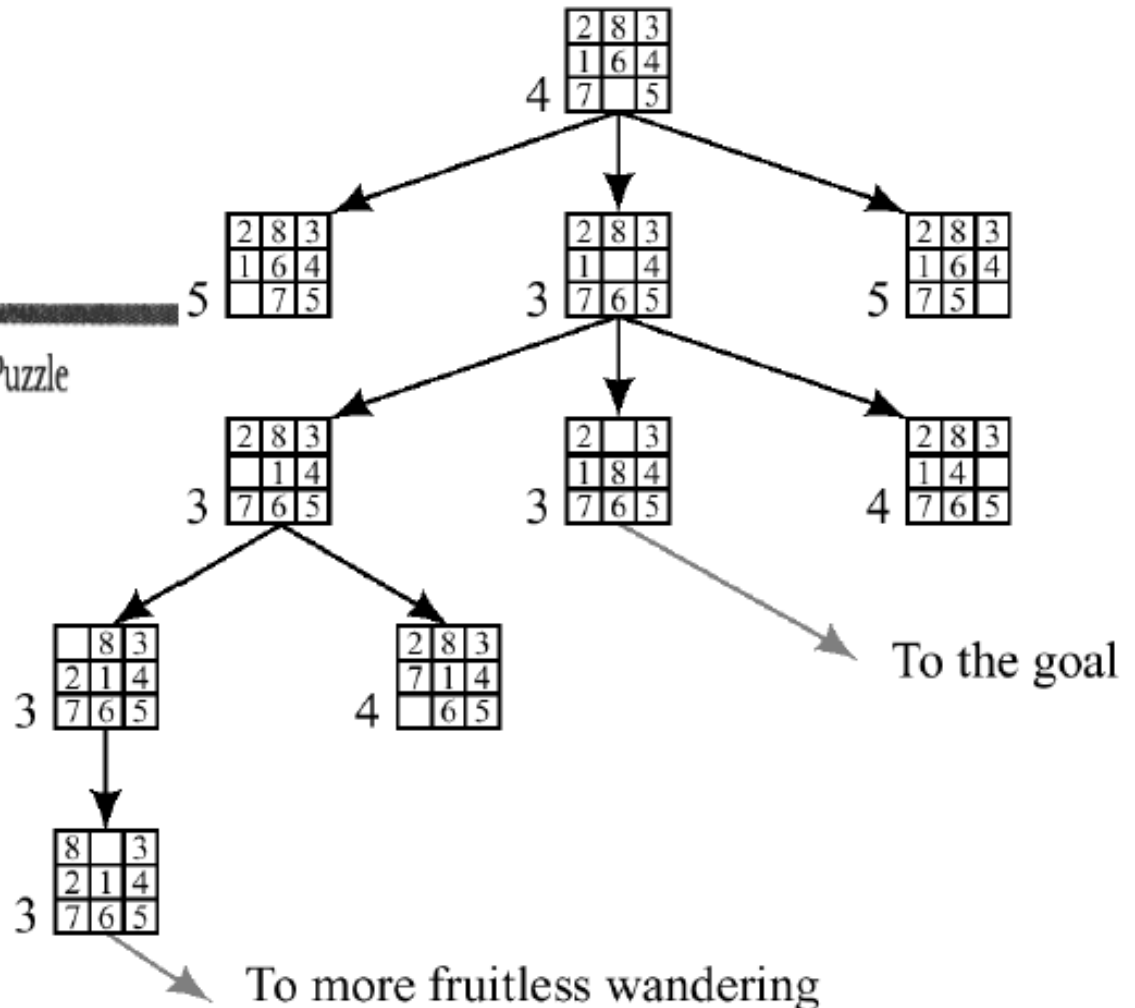
Best-First (Greedy) Search:
 $f(n)$ = number of misplaced tiles

2	8	3
1	6	4
7		5

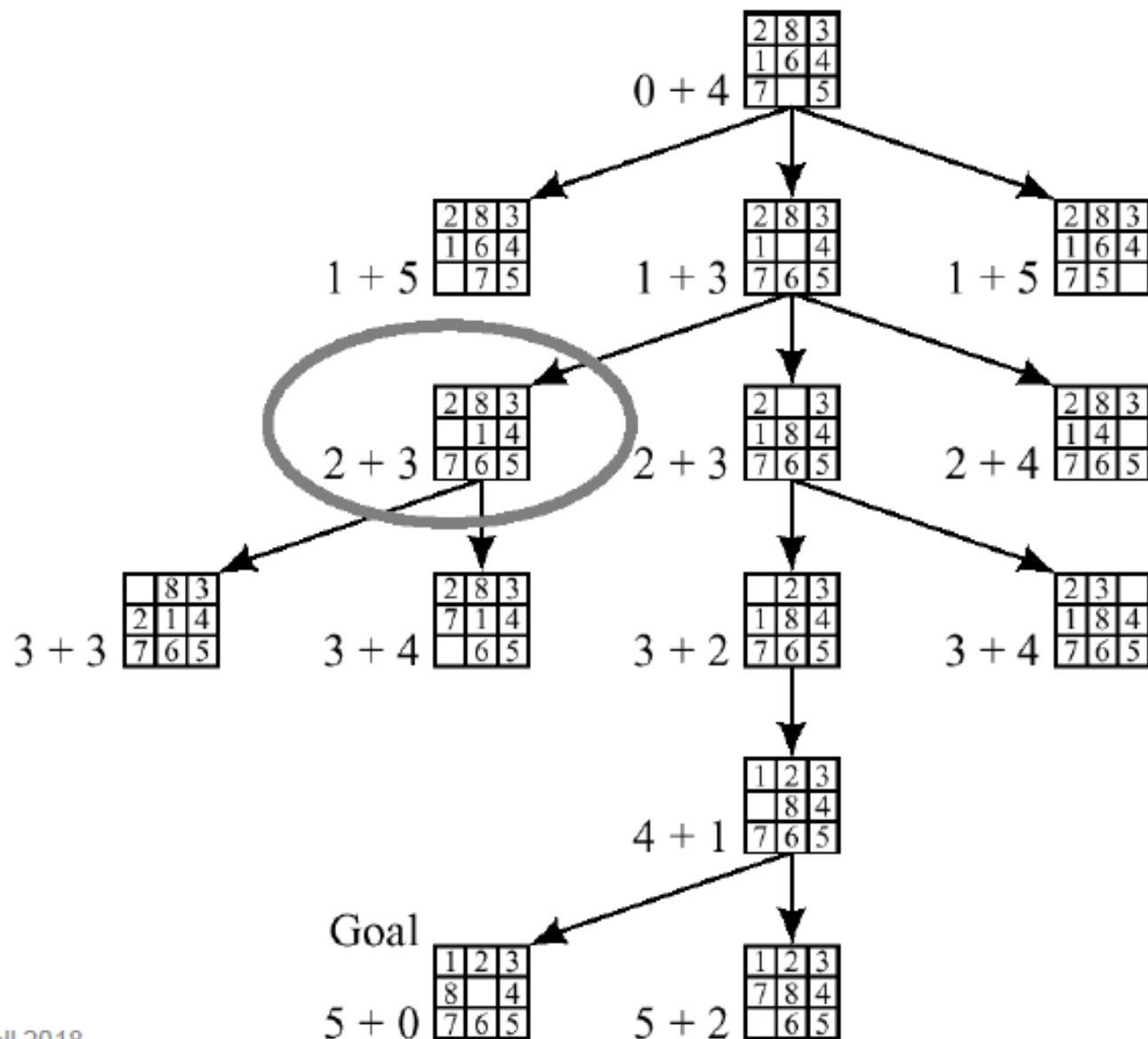
1	2	3
8		4
7	6	5

Figure 8.1

Start and Goal Configurations for the Eight-Puzzle



A* on 8-Puzzle with $h(n) = \#$ misplaced tiles



算法 7: A* 搜索

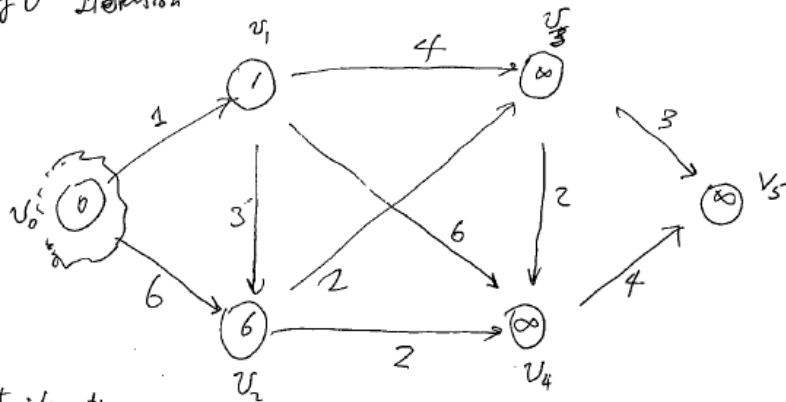
- 1: 初始化以 f 为键值的优先级队列 q , 令其为空; $q = \text{queue of nodes to be processed}$
- 2: 初始化访问表 v , 令其为空; $v = \text{list of visited nodes (similar to set } S \text{ in Dijkstra alg.)}$
- 3: 将初始节点 S 放入优先级队列 q , $g(S) = 0, f(S) = h(S)$;
- 4: **while** 优先级队列 q 非空 **do**
- 5: 弹出队首 n , 并将 n 加入访问表 v ;
- 6: **if** n 为目标节点 **then**
- 7: 返回成功;
- 8: L 是 n 可转移到且不在优先级队列 q 和访问表 v 中的后继节点的集合, M 是 n 可转移到且在优先级队列 q 中的后继节点的集合;
- 9: **for** $m \in L$ **do**
- 10: $g(m) = g(n) + c(n, m)$; n is the predecessor of m
- 11: $f(m) = g(m) + h(m)$;
- 12: 将 m 加入队列;
- 13: **for** $m \in M$ **do**
- 14: $g(m) = g(n) + c(n, m)$;
- 15: $f(m) = g(m) + h(m)$;
- 16: **if** 新的 $f(m) < q$ 中的 $f(m)$ **then**
- 17: 更改 m 的父节点为 n ; Predecessor of m
- 18: 更新 q 中的 $f(m)$;
- 19: 返回失败;

nodes adjacent to n

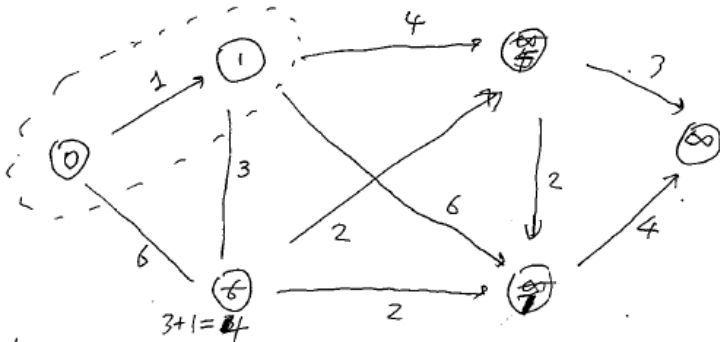
细心的读者会发现, 当 A* 搜索算法中所有节点的启发函数值 $h(n) = 0$ 时, 算法即为 Dijkstra 算法。

A* Search: Modification of Dijkstra Algorithm

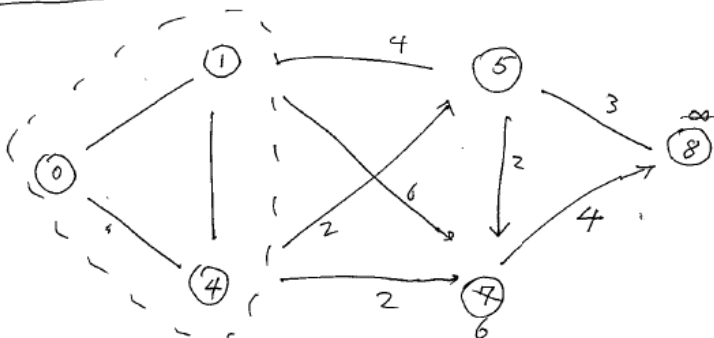
End of 0th Iteration



End of 1st iteration



end of 2nd iteration



From "Introduction to Algorithms"

DIJKSTRA(G, w, s)

```

1 INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  $S = \emptyset$ 
3  $Q = G.V$ 
4 while  $Q \neq \emptyset$ 
5      $u = \text{EXTRACT-MIN}(Q)$ 
6      $S = S \cup \{u\}$ 
7     for each vertex  $v \in G.\text{Adj}[u]$ 
8         RELAX( $u, v, w$ )
    
```

Here use the $f(n)$ function

update

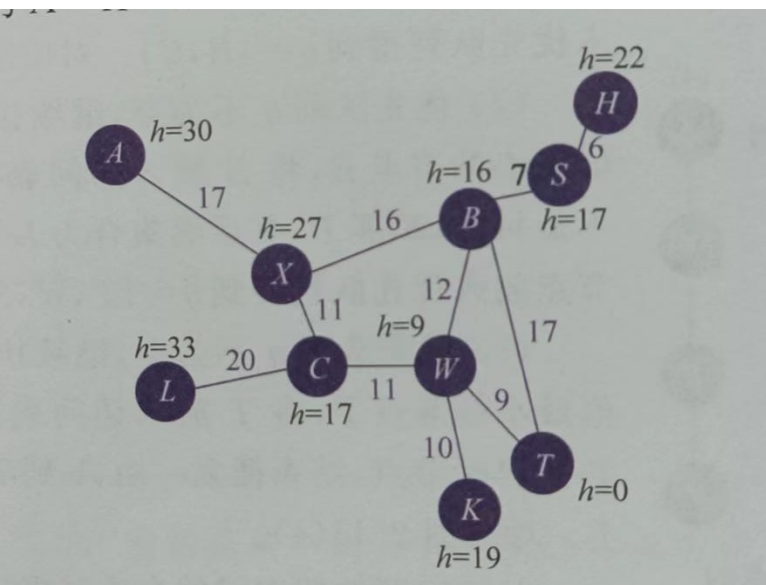
RELAX(u, v, w)

```

1 if  $v.d > u.d + w(u, v)$ 
2      $v.d = u.d + w(u, v)$ 
3      $v.\pi = u$ 
    
```

Here use $\min f(n) = g(n) + h(n)$

Homework A star 1: Running A* on the graph. A is start, T is goal. Your task: add $g(n)$ and $f(n)$ computation in each iteration of the while loop. See A* algorithm details.



初始化时, 优先队列为空 $q = \{\}$, 访问表为空 $v = \{\}$ 。算法执行过程如下:

(1) 将初始节点 A 放入优先队列得到 $q = \{A\}$ 。

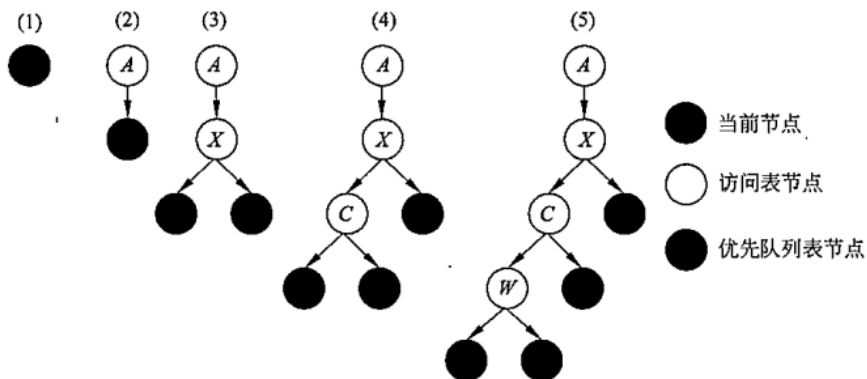
(2) 优先队列 q 不为空, 继续执行。弹出优先队列中 f 值最小的节点 A, 将 A 放入访问表得到 $v = \{A\}$ 。A 不是目标, 扩展 A, 其后继集合为 $\{X\}$ 。将后继集合中的节点放入优先队列得到 $q = \{X\}$ 。对应图 2.12(1)。

(3) 优先队列 q 不为空, 继续执行。弹出优先队列中 f 值最小的节点 X, 将 X 放入访问表得到 $v = \{A, X\}$ 。X 不是目标, 扩展 X, 其后继集合为 $\{B, C\}$ 。将后继集合中的节点放入优先队列得到 $q = \{B, C\}$ 。对应图 2.12(2)。

(4) 优先队列 q 不为空, 继续执行。弹出优先队列中 f 值最小的节点 C, 将 C 放入访问表得到 $v = \{A, X, C\}$ 。C 不是目标, 扩展 C, 其后继集合为 $\{L, W\}$ 。将后继集合中的节点放入优先队列得到 $q = \{W, B, L\}$ 。对应图 2.12(3)。

(5) 优先队列 q 不为空, 继续执行。弹出优先队列中 f 值最小的节点 W, 将 W 放入访问表 $v = \{A, X, C, W\}$ 。W 不是目标, 扩展 W, 其后继集合为 $\{T, K\}$ 。将后继集合中的节点放入优先队列得到 $q = \{T, B, K, L\}$ 。对应图 2.12(4)。

(6) 优先队列 q 不为空, 继续执行。弹出优先队列中 f 值最小的节点 T, 将 T 放入访问表得到 $v = \{A, X, C, W, T\}$ 。T 为目标节点, 结束搜索。由 A 到 T 的路径为 $A \rightarrow X \rightarrow C \rightarrow W \rightarrow T$ 。对应图 2.12(5)。

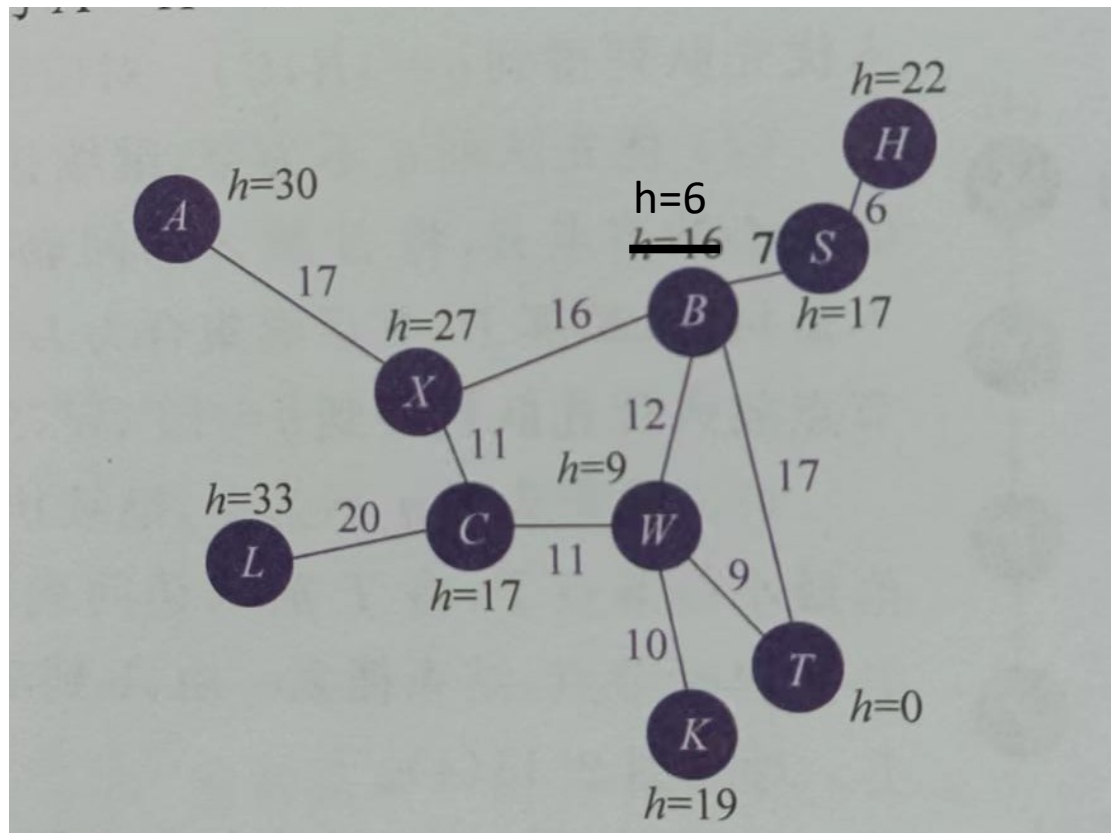


Homework A star 2: Running A* on the graph. A is start node, T is goal node. Here on node B, $h=16$ has been changed to $h=6$.

Can you get the same optimal path from A to T irrespective of whether $h=16$ or $h=6$ on node B? They are both admissible.

Give details as in **Homework A star 1**.

Is there some problem with the algorithm?



Homework: AStar3

Homework:

Problem 3.7 on page 114 of the textbook. Solve two questions.

- (a) How many states in this search problem?
- (b) Why shortest path goes along the edge of a polygon?

